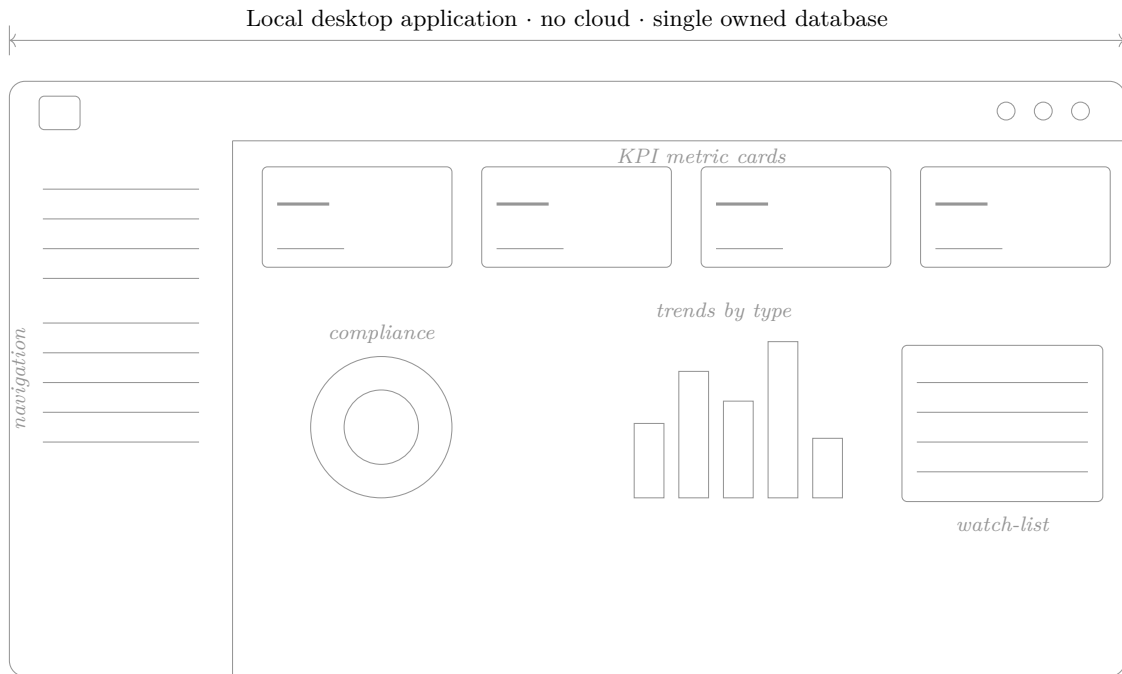


ERP-Contractor

A Local-First QHSE Operations Platform

Systems Design, Data Architecture, Compliance Engine, and Reporting



General arrangement of the primary dashboard (schematic, not a live capture).

Author: Salman K. Alwajaan
Contact: Salman.khalid.alwajaan@gmail.com
Document: Revision 1.0 · 2026

Abstract

ERP-Contractor is a locally hosted, single-owner software platform that unifies the Quality, Health, Safety, Environment (QHSE) and human-resources operations of a small electrical-contracting company. It replaces a fragmented collection of spreadsheets with one relational database and a dynamic desktop interface, eliminating manual reconciliation while adding proactive compliance monitoring. This report documents the platform’s design goals, software architecture, relational data model, functional modules, the tag-based (NFC / QR) inspection system, PPE inventory and storage, the credential compliance (expiry) engine, spreadsheet interoperability, security and data-ownership model, and verification results. The system is implemented as an Electron desktop application (React user interface) backed by an embedded SQLite database, and runs entirely offline on a single laptop with no cloud dependency or subscription.

Contents

1	Introduction	3
1.1	Background and Problem Statement	3
1.2	Objectives	3
1.3	Scope	3
2	System Overview	3
2.1	Design Goals	3
2.2	Technology Stack	4
3	Software Architecture	4
3.1	Process Model	4
3.2	The Data Channel	4
3.3	Persistence	4
4	Data Model	5
4.1	Relational Schema	5
4.2	Entity Relationships	6
4.3	Audit Trail	6
5	Functional Modules	6
5.1	People and Compliance	7
5.2	Safety	7
5.3	Environment	7
5.4	Quality	7
5.5	Projects and Executive Dashboard	8
6	NFC-Based Inspection System	8
6.1	Tag-to-Asset Workflow	8
6.2	Checklist Templates	8
6.3	Scoring and Result	9
6.4	Two-Stream Reporting and Offline Operation	9

7	Stock, Inventory and PPE Storage	9
7.1	PPE Catalogue and Stock Levels	9
7.2	Issue, Acknowledgment and Return	9
7.3	Equipment Storage and Asset Registry	10
7.4	Inventory Integrity	10
8	Work-Order and Task Management	10
8.1	Task Structure	10
8.2	Lifecycle and States	10
8.3	Priority and Overdue Detection	11
8.4	Cross-Module Origination	11
8.5	Executive Roll-Up	11
9	Compliance and Expiry Engine	11
9.1	Status Classification	11
9.2	Watch-list and Alerts	12
10	Data Interoperability	12
11	Security, Access and Data Ownership	12
12	Deployment and Operation	12
13	Verification and Results	13
14	Limitations and Future Work	13
14.1	Current Limitations	13
14.2	Roadmap	13
14.3	Extensibility Note	14
15	Conclusion	14
A	Key Performance Indicators	14

1 Introduction

1.1 Background and Problem Statement

The company operates as a contractor for a national electrical utility. Critical operational data — employee records, professional licenses, training courses, government and client-issued identity cards (Iqama, Industrial Security Cards, Municipality permits), personal protective equipment (PPE), daily safety inspections, ISO documentation, environmental logs, quality records, and project work orders — was historically distributed across numerous, disconnected Excel files. This manual model is time-consuming, error-prone, and offers no real-time visibility. Because the utility client mandates weekly updates of employee data and because lapsed credentials carry regulatory and contractual risk, the absence of a single source of truth is a direct operational liability.

1.2 Objectives

- Consolidate all QHSE and HR data into one relational database (a single source of truth).
- Provide a fast, modern, desktop interface — explicitly not a generic spreadsheet clone.
- Track every expiring credential automatically and surface it before it lapses.
- Preserve full data ownership: local storage, offline operation, no subscription.
- Interoperate with the existing Excel workflow through structured import and export.
- Extend beyond safety to dedicated Environment and Quality departments.

1.3 Scope

This document covers the delivered version 1.0 of the platform: its architecture, data model, modules, and operation. Hardware integration for physical NFC tags (a mobile-companion concern) and a signed, packaged application binary are identified as future work in Section 14.

2 System Overview

2.1 Design Goals

Four properties drove every design decision: (i) **local-first** — the application and its data reside on the user's machine; (ii) **owned** — data is stored in an open, portable database file the user controls; (iii) **powerful UI** — a dynamic, themeable interface with charts and drill-down, in both dark and bright modes; and (iv) **single platform** — one integrated system rather than a patchwork of tools.

2.2 Technology Stack

Layer	Technology	Role
Desktop shell	Electron	Native window, offline runtime, OS integration
User interface	React + Vite	Dynamic, component-based front end
Styling / theming	Tailwind CSS	Design tokens, dark / bright mode
Charts	Recharts	KPI donut, trend and bar visualizations
Database	SQLite (via sql.js)	Embedded relational store, single file
Spreadsheets	ExcelJS	Excel / CSV import and export

Table 1: Implementation stack. The SQLite engine is compiled to WebAssembly, avoiding native build steps while remaining a genuine relational database.

3 Software Architecture

3.1 Process Model

The application follows Electron’s isolated three-part model. The *renderer* hosts the React interface and never touches the database or file system directly. A *preload* bridge exposes a single, narrow, contextually isolated function to the interface. The *main* process owns the database, the file system, and all native dialogs. This separation is a security boundary: the interface can only request data through one audited channel.

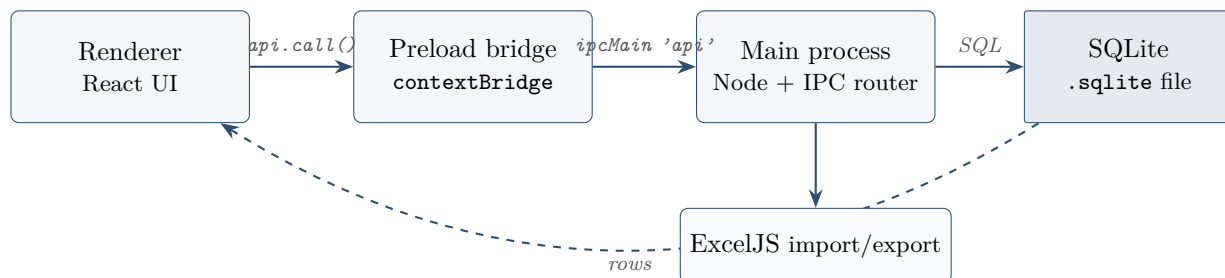


Figure 1: Process architecture. All data flows through one inter-process channel; the renderer has no direct database or file-system access.

3.2 The Data Channel

Every operation is expressed as a single call of the form `call(action, params)` where `action` is a dotted string such as `employees.list` or `licenses.create`. The main process routes the action to a generic create/read/update/delete engine or to a specialised handler (dashboard aggregation, compliance scan, Excel transfer). This uniformity keeps the surface area small and every write auditable.

3.3 Persistence

The database is a single SQLite file stored in the operating system’s per-user application-data directory. After each mutation the engine serialises the full database to disk, guaranteeing durability; the identifier of a newly inserted row is captured before serialisation. Backing up the system is a single file copy.

4 Data Model

4.1 Relational Schema

The schema is normalised around a central **employees** table. Each employee is the anchor for their licenses, cards, courses and issued PPE, linked by the business key **emp_id**. Project work, environmental records and quality reports form a second cluster linked by project. Joined database *views* carry human-readable names (employee, project) into list queries so the interface never needs to resolve foreign keys itself.

Table	Purpose	Key link
employees	Master personnel records (single source of truth)	—
id_cards	Iqama, Industrial Security, Municipality, etc.	emp_id
licenses	Professional / operating licenses with expiry	emp_id
courses	Completed training and certifications	emp_id
ppe_items	PPE catalogue and stock levels	—
ppe_issues	PPE assigned to an employee (with acknowledgment)	emp_id
assets	Job sites, vehicles, equipment (NFC / QR tags)	—
inspections	Daily inspections with checklist and score	asset_id
documents	Controlled ISO documents (45001 / 14001 / 9001)	—
document_versions	Version history for each document	document_id
projects	Contracts, orders and study projects	—
tasks	Work orders and assignments	project_id
environment_records	Cooling-oil pulls, staking returns, waste removal	project_id
quality_ncr	Non-conformance reports and corrective actions	project_id
quality_audits	Internal / external / certification audits	—
audit_log	Immutable record of every create / update / delete	—

Table 2: Core relational tables. Reference tables (users, settings) are omitted for brevity.

4.2 Entity Relationships

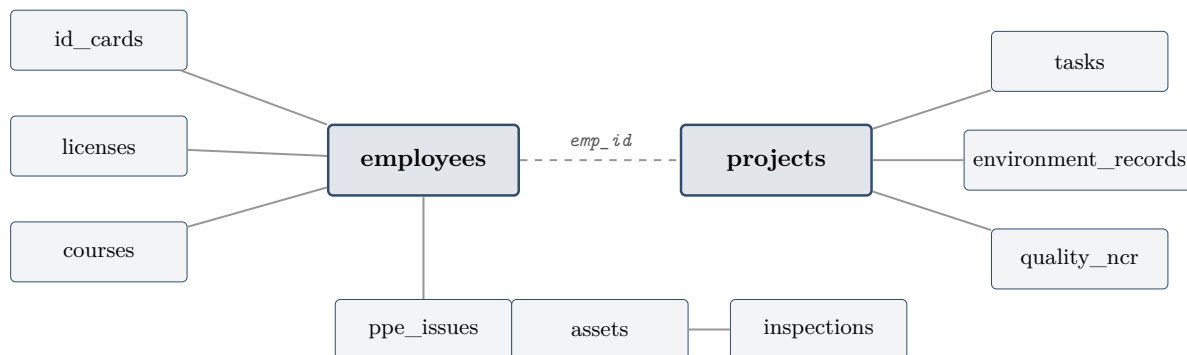


Figure 2: Entity relationships. The employee record is the hub for all personnel credentials; a second cluster organises project delivery, environment and quality.

4.3 Audit Trail

Every create, update and delete is written to an append-only `audit_log` table capturing the acting user, the action, the affected entity and a timestamp — the evidentiary trail required for ISO 45001 and client audits.

5 Functional Modules

The interface is organised into department-aligned modules reached from a persistent sidebar.

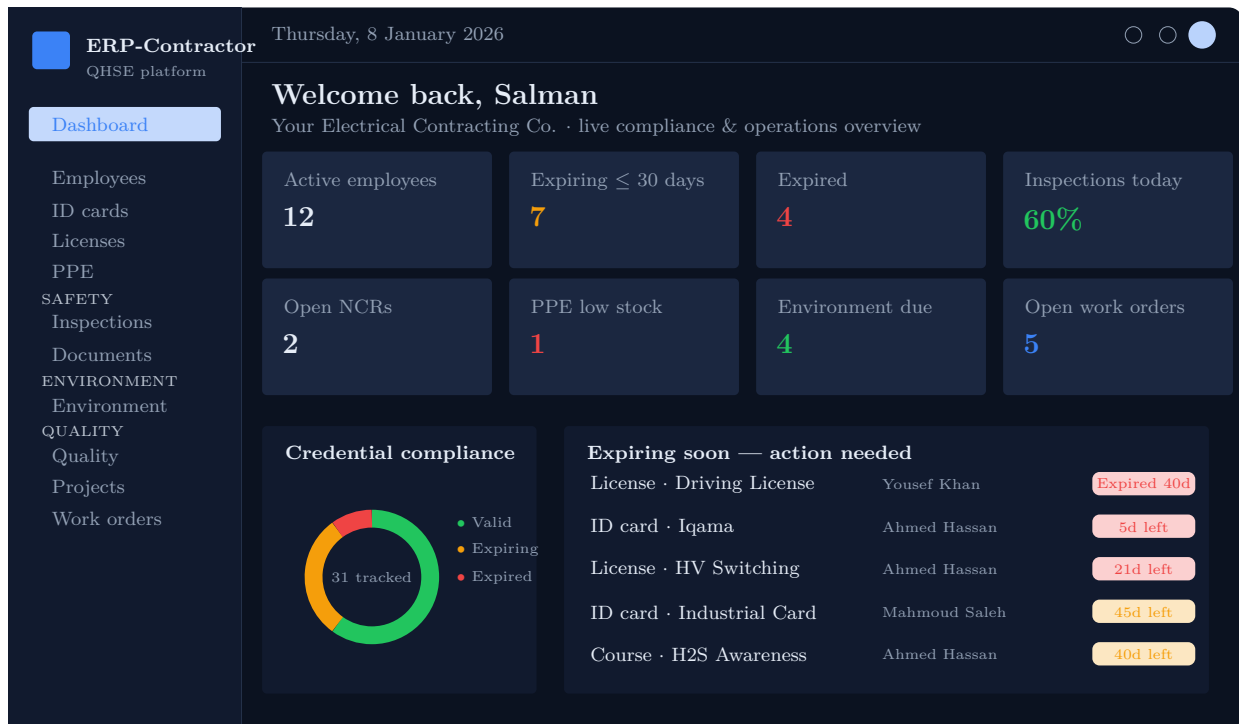


Figure 3: The executive dashboard rendered in the application’s dark theme, populated with the platform’s seeded demonstration data. Key indicators, the credential-compliance breakdown and the “expiring soon” watch-list are visible at a glance.

5.1 People and Compliance

Employee master records, with a profile view that gathers every linked ID card, license, course and PPE issue in one place. PPE is tracked both as catalogue stock and as items issued to individuals with an acknowledgment flag; inventory and storage are detailed in Section 7.

5.2 Safety

Daily inspections are captured against a tagged asset (job site, vehicle or equipment). Selecting the asset loads the appropriate checklist template; the module computes a score and a pass / conditional / fail result and records proof of presence. A controlled document register manages ISO 45001 material with full version history. The tag-based inspection workflow is detailed in Section 6.

5.3 Environment

A dedicated register for transformer cooling-oil pulls, staking returns and waste-removal schedules, supporting the “claimant” extractor reports required by the client.

5.4 Quality

Non-conformance reports (NCRs) with root cause and corrective action, plus an audit register covering internal, external and certification audits under ISO 9001.

5.5 Projects and Executive Dashboard

Projects carry progress and status; work orders are assigned to employees with priority and due dates (work-order management is detailed in Section 8). The dashboard (Figure 3) aggregates indicators from every module into a single executive overview for the CEO.

6 NFC-Based Inspection System

The safety module replaces unreliable paper logs with tag-based digital inspections that yield verifiable proof of presence. Durable tags — NFC, QR or barcode — are fixed to every job site, vehicle and item of equipment and registered against an asset record, so a single scan identifies exactly what is being inspected.

6.1 Tag-to-Asset Workflow

1. **Placement.** Tamper-resistant tags are mounted on trucks, cars, job-site boards and equipment; each is registered in **assets** with its identifier (license plate / site ID) and tag value.
2. **Scan / select.** The officer scans (or selects) the tag; the application resolves it to the asset and auto-populates identifier, type and location.
3. **Data segregation at source.** The asset type routes the record into one of two streams — *Site Safety* or *Fleet* — without any manual sorting.
4. **Checklist.** A type-specific template loads; each line is marked OK, Not OK or N/A with an optional remark.
5. **Proof of presence.** The inspector, timestamp and tag / asset identifier are recorded, giving an auditable record that the inspection physically took place.

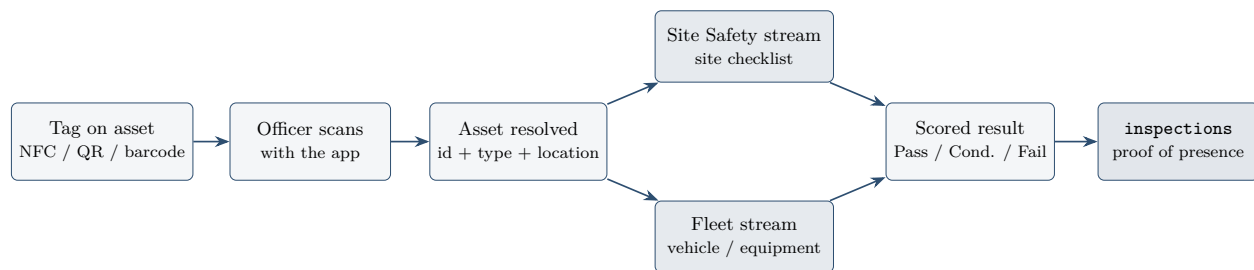


Figure 4: Tag-based inspection flow. The identifier captured at scan time routes each record into the Site Safety or Fleet stream before it is scored and stored — two operational views from one table.

6.2 Checklist Templates

Three built-in, extensible templates cover the common cases:

- **Site Safety** — PPE worn correctly, work area barricaded, permit to work valid, earthing and isolation applied, tools inspected, emergency arrangements, housekeeping.
- **Vehicle** — tyres and brakes, lights and indicators, warning beacon, first-aid and extinguisher, vehicle documents (Istimara), body and hydraulics, driver license valid.
- **Equipment** — guards and safety devices, inspection sticker valid, earthing / RCD tested, no visible damage, operating manual available.

6.3 Scoring and Result

Let n_{ok} and n_{no} be the number of OK and Not-OK items (N/A items are excluded). The score is

$$s = \text{round}\left(100 \frac{n_{\text{ok}}}{n_{\text{ok}} + n_{\text{no}}}\right) \%,$$

and the result is **Pass** when $n_{\text{no}} = 0$; **Conditional** when $n_{\text{no}} > 0$ and $s \geq 70$; otherwise **Fail**. A failed inspection flags the asset for follow-up and can raise a work order in the projects module.

6.4 Two-Stream Reporting and Offline Operation

Because the identifier captured at scan differs by asset type (site ID versus license plate), every inspection lives in one table yet filters cleanly into a Site Safety view (by job site) and a Fleet view (by vehicle / equipment) — delivering the two platforms the department needs with no data duplication. For disconnected field work, a mobile companion (future work, Section 14) caches inspections and synchronises on reconnection; each inspection is already stored as a structured record with its full checklist.

7 Stock, Inventory and PPE Storage

The platform manages both the *store* of protective equipment and its *allocation* to individuals, giving a continuous, reconciled audit of who holds what and how much remains on hand.

7.1 PPE Catalogue and Stock Levels

Every catalogue item carries a category, unit, available sizes, a current stock level and a minimum level. When stock reaches or falls below the minimum, the item is flagged and raises the dashboard “PPE low stock” indicator, prompting reorder.

Item	Category	Unit	Sizes	Stock	Min	Status
CAT-1 Insulating Helmet	Head	pcs	M/L/XL	4	6	Low
Arc Flash Suit 8 cal	Body	set	M/L/XL	9	4	OK
Insulating Gloves Class 0	Hand	pair	9/10/11	12	8	OK
Safety Glasses	Eye	pcs	Std	40	15	OK
Safety Boots S3	Foot	pair	40–44	18	10	OK
Hi-Vis Vest	Body	pcs	M/L/XL	30	12	OK

Table 3: Seeded PPE catalogue. An item is flagged **Low** when stock $\leq$ minimum; here the CAT-1 helmet (4 of a 6 minimum) drives the low-stock alert.

7.2 Issue, Acknowledgment and Return

When equipment is issued, the system records the employee, item, size, quantity, issue date and a re-test / expiry date, and requires an *acknowledgment* — the worker signs for the item. Each issue carries a status: **Issued**, **Returned**, **Lost** or **Expired**. An employee’s profile shows the complete history of equipment they hold, and any dated item (for example insulating gloves requiring periodic re-test) flows into the compliance engine exactly like an expiring credential.

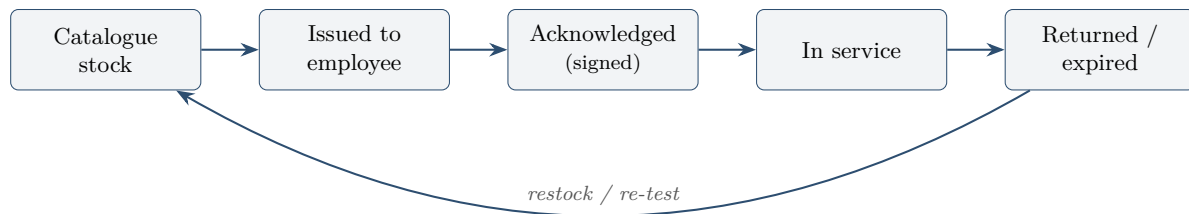


Figure 5: PPE lifecycle. Returned items replenish stock; dated items re-enter the compliance engine at their re-test date.

7.3 Equipment Storage and Asset Registry

Beyond consumable PPE, stored equipment — generators, cable rigs, tools — is registered in the **assets** table with its own tag, storage location and status. Stored items are therefore inventoried, inspectable through the same NFC / QR workflow (Section 6), and traceable to a physical location at all times.

7.4 Inventory Integrity

Because stock levels and issue history share one relational store, the on-hand count and the allocation record reconcile automatically — there is no separate spreadsheet to drift out of sync. Every stock adjustment and issue is written to the **audit_log**, providing a defensible inventory trail.

8 Work-Order and Task Management

Field and office work is coordinated through work orders (tasks) attached to projects, turning the platform from a passive record-keeper into an active operational coordination tool.

8.1 Task Structure

Each work order carries a parent project, a title, an assignee drawn from the employee master, a priority (**Low**, **Medium**, **High**, **Urgent**), a status, a due date and a free-text description. Because the assignee is a real employee record, that person’s licenses, competencies and issued PPE are one click away when work is allocated — a supervisor can avoid assigning, say, high-voltage work to someone whose HV license has lapsed.

8.2 Lifecycle and States

A work order moves through four states. A task whose due date has passed while it is not yet **Done** is additionally flagged *overdue* in red, reusing the compliance colour language so attention is drawn consistently across the whole platform.

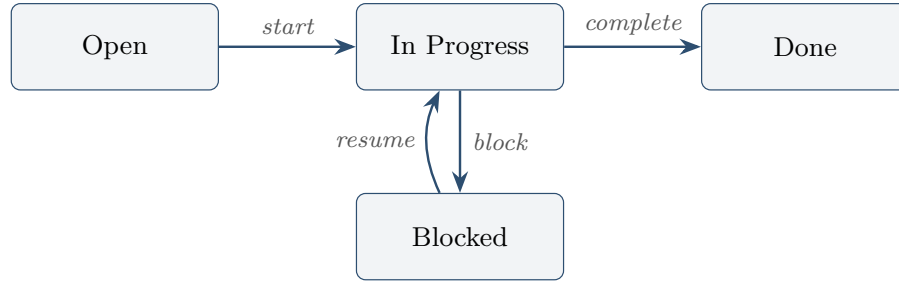


Figure 6: Work-order state model. Priority sets the colour (Urgent / High red, Medium amber, Low grey); an overdue task is flagged regardless of priority.

8.3 Priority and Overdue Detection

Priority is colour-coded for instant triage, and due dates reuse the day-difference logic of the compliance engine: a task with due date $d < 0$ that is not **Done** is overdue. The dashboard’s “Open work orders” indicator counts every task not yet **Done**.

8.4 Cross-Module Origination

Crucially, work orders are not created only by hand — they close the loop between detection and action:

- a **failed inspection** (Section 6) raises a corrective work order against the asset;
- an **open NCR** in quality spawns corrective-action tasks with owners and due dates;
- a **low-stock PPE** item can trigger a reorder task.

8.5 Executive Roll-Up

Individual tasks aggregate upward: project progress is derived from task completion, and the executive dashboard summarises open work, overdue items and completion rates so management sees operational health without chasing individual updates.

9 Compliance and Expiry Engine

9.1 Status Classification

Every credential with an expiry date is classified daily. Let d be the number of days until expiry. The status is

$$\text{status}(d) = \begin{cases} \text{expired} & d < 0, \\ \text{critical} & 0 \leq d \leq 30, \\ \text{warning} & 30 < d \leq 90, \\ \text{valid} & d > 90, \end{cases}$$

rendered as red, red, amber and green respectively. The thresholds echo the client’s escalating notification points.

9.2 Watch-list and Alerts

A single union query gathers every dated credential — ID cards, licenses, courses and PPE — computes d , and sorts ascending. The result drives the dashboard watch-list, the alert badge count, and the alerts panel, converting a reactive risk (a lapsed credential discovered too late) into a managed, visible process.

10 Data Interoperability

Because the existing workflow is spreadsheet-based, every module exports to a formatted Excel workbook. Personnel and credential tables additionally support import: column headers are matched case-insensitively to database fields, and employees are *upserted* by `emp_id` so a weekly refresh updates existing people rather than duplicating them. The import path acts as a validation gate rather than a blind copy, protecting the integrity of the single source of truth.

11 Security, Access and Data Ownership

- **Local ownership.** Data lives in one SQLite file under the user’s control; there is no cloud service and no third party with a copy.
- **Isolation.** The interface reaches data only through the single preload bridge; it holds no database or file-system handles.
- **Credentials.** Application logins are stored salted and hashed (scrypt), never in plain text.
- **Audit trail.** The append-only `audit_log` records who changed what and when.
- **Roles.** A role field is retained on every account for future least-privilege policies; the delivered configuration grants full edit to all operators by request.

12 Deployment and Operation

Concern	Detail
Launch	<code>npm run dev</code> , or the bundled desktop launch shortcut
Runtime	Electron desktop window; Vite dev server on a dedicated local port
Data location	Per-user application-data directory (single <code>.sqlite</code> file)
Backup	Copy the single <code>.sqlite</code> file
Offline	Fully functional with no network connection

Table 4: Operational summary.

13 Verification and Results

Check	Result
Front-end production build	Compiles cleanly (2,205 modules, 0 errors)
Database schema and seeding	12 employees and all module data created
Authentication	Valid login accepted; invalid rejected
KPI aggregation	Correct counts (e.g. 7 expiring, 4 expired)
Compliance union query	Correct day-to-expiry and ordering
Create / read / update / delete	Verified across modules
Excel import / export	Round-trips verified
Application boot	Launches, initialises database, no errors
Persistence	Data survives restart (no re-seed)

Table 5: Verification summary for version 1.0.

14 Limitations and Future Work

14.1 Current Limitations

- **Physical NFC capture.** The data model is tag-ready, but live NFC taps in the field require a mobile companion; the desktop client uses tag-ID / selection and manual entry.
- **Packaged binary.** The system runs from the development launcher; a signed, double-click application bundle is a straightforward next step.
- **Single workstation.** The database is local to one machine; concurrent multi-user access is not yet supported.
- **In-app notifications only.** Expiry alerts surface inside the application; email, SMS and Teams delivery are not yet wired.
- **English interface.** An Arabic (right-to-left) interface is not yet provided.

14.2 Roadmap

The architecture is deliberately extensible: one relational store and one uniform data channel mean new modules and integrations attach without disturbing what already works. Growth is planned in three phases.

Phase	Theme	Key deliverables
Phase 2	Field & mobility	Mobile companion for live NFC / QR taps with offline caching and sync; camera-based scanning; signed, double-click desktop binary; Arabic (RTL) localisation.
Phase 3	Automation & integration	Scheduled PDF / Excel client reports emailed automatically; email / SMS / Teams expiry alerts; granular role-based access (least privilege); integration hooks to government portals (Iqama / Muqem, Municipality) and payroll / HRIS; one-click weekly client data export.
Phase 4	Intelligence & scale	Predictive safety analytics that flag high-risk assets and sites from inspection history; optional encrypted multi-device sync and backup; forecasting dashboards; e-signature for PPE acknowledgment and applications; audit-ready compliance report packs.

Table 6: Phased roadmap. Each phase builds on the delivered version 1.0 core without re-architecting it.

14.3 Extensibility Note

Because every module is a table plus a configuration entry, adding a new register — for example a training-matrix or a permit-to-work log — is largely declarative. The retained per-account role field activates least-privilege access whenever the organisation is ready, and the Excel import / export path provides an immediate bridge to any external system pending a deeper integration.

15 Conclusion

ERP-Contractor replaces a fragile, spreadsheet-based process with a single, owned, offline platform that unifies safety, environment, quality, personnel and project operations. Its relational core, proactive compliance engine and department-aligned modules convert scattered records into a coherent, auditable system — delivering immediate operational visibility while remaining entirely under the user’s control.

A Key Performance Indicators

Indicator	Definition
Active employees	Count of employees with status Active
Expiring ≤ 30 d	Credentials with $0 \leq d \leq 30$
Expired	Credentials with $d < 0$
Inspections today	Completed today $\div$ active sites and vehicles
Open NCRs	Non-conformance reports not Closed
PPE low stock	Catalogue items at or below minimum level
Environment due	Environmental records not Completed
Open work orders	Tasks not Done

Table 7: Dashboard indicators and their definitions.